DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC SESSION: 2023–2027

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

RENTRIDE — MULTI-VENDOR VEHICLE RENTAL MARKETPLACE

Standard IEEE-Inspired Academic Software Requirements Specification for Major Project Submission

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology (B.Tech)
in
Computer Science and Engineering

SUBMITTED BY:

Student Name: _____

Enrollment No: _____

Roll Number: _____

UNDER THE SUPERVISION OF:

Project Guide: _____

Designation: Assistant Professor / Professor

Department of Computer Science & Engg.

Semester: VIII (Final Year Major Project)

Institution: _____

MAY 2026

TABLE OF CONTENTS

1. INTRODUCTION **1**

1.1 Purpose 1

1.2 Scope of the System 1

1.3 Intended Audience & Reading Suggestions 2

1.4 Product Overview 2

1.5 Definitions, Acronyms and Abbreviations 3

1.6 References 3

2. OVERALL DESCRIPTION **4**

2.1 Product Perspective 4

2.2 Product Functions Summary 4

2.3 User Classes and Characteristics 5

2.4 Operating Environment 6

2.5 Design and Implementation Constraints 6

2.6 Assumptions and Dependencies 7

3. SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS **8**

3.1 Module A: Authentication & Authorization (FR-AUTH) 8

3.2 Module B: Customer Profile & KYC (FR-CUST) 9

3.3 Module C: Vendor Onboarding & Fleets (FR-VEND) 10

3.4 Module D: Vehicle Management (FR-VEH) 11

3.5 Module E: Vehicle Categories (FR-CAT) 12

3.6 Module F: City Search & Filter Engine (FR-SRCH) 13

3.7 Module G: Collision & Availability Engine (FR-AVAL) 14

3.8 Module H: Rental Plans (Hourly, 12h, 24h) (FR-PLAN) 15

3.9 Module I: Booking Lifecycle & Locks (FR-BOOK) 16

3.10 Module J: Server Pricing Engine (FR-PRIC) 17

3.11 Module K: Coupons & Promotions (FR-COUP) 18

3.12 Module L: Payment Processing via Razorpay (FR-PAY) 19

3.13 Module M: Walk-In Offline Bookings (FR-WKIN) 20

3.14 Module N: Vehicle Handover & Return (FR-TRIP) 21

3.15 Module O: Maintenance Management (FR-MAINT) 22

3.16 Module P: Admin Governance & Moderation (FR-ADM) 23

3.17 Module Q: Customer Reviews & Ratings (FR-REV) 24

3.18 Module R: In-App Notifications (FR-NOTIF) 25

3.19 Module S: AI Recommendations Engine (FR-AI-REC) 26

3.20 Module T: AI 7-Day Demand Forecasting (FR-AI-FCST) 27

3.21 Module U: AI Dynamic Pricing Engine (FR-AI-PRIC) 28

3.22 Module V: Master Vehicle Catalog (FR-CATL) 29

4. NON-FUNCTIONAL REQUIREMENTS (NFR) 30

5. SYSTEM ARCHITECTURE & DATA FLOW DIAGRAMS 33

6. USE CASE MODEL & SPECIFICATIONS (UC-01 to UC-17) 35

7. DATA REQUIREMENTS & ENTITY-RELATIONSHIP MODEL 40

8. DATA DICTIONARY 42

9. REST API SPECIFICATIONS 45

10. SECURITY REQUIREMENTS & IMPLEMENTATION 48

11. BOOKING & BUSINESS LOGIC SPECIFICATION 50

12. AI / ML MICROSERVICE DETAILED SPECIFICATION 52

13. USER INTERFACE SPECIFICATIONS 54

14. ERROR HANDLING & EXCEPTION MATRIX 55

15. TESTING REQUIREMENTS & VERIFICATION SUITE 56

16. DEPLOYMENT & INFRASTRUCTURE 58

17. SYSTEM LIMITATIONS & KNOWN CONSTRAINTS 59

18. DEVELOPMENT ROADMAP & FUTURE SCOPE 60

19. CONCLUSION & SIGNOFF 61

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional and non-functional requirements for the **RentRide Multi-Vendor Vehicle Rental Marketplace**. It establishes a formal, verifiable contract between system stakeholders—including software engineers, platform architects, academic project evaluators, and prospective marketplace operators. The document specifies system behavior, constraints, algorithmic logic, interface designs, and operational data flows.

1.2 Scope of the System

RentRide is an enterprise-ready, cloud-native web platform that coordinates short-term vehicle rentals across two-wheelers (scooters, commuter motorcycles, electric vehicles) and four-wheelers (hatchbacks, sedans, SUVs). The software orchestrates customer discovery, real-time collision-free availability checking, tiered rental plans, digital payments, vendor fleet administration, offline walk-in bookings, physical trip custody (odometer/OTP), and machine learning services for recommendations, demand forecasting, and dynamic pricing.

1.3 Intended Audience & Reading Suggestions

This document is structured for the following stakeholders:

- **Academic Evaluation Committee & Project Guides:** Focus on Sections 1 to 6 (System Architecture, Requirements, Use Cases) and Section 11 (Business & Scheduling Logic).
- **Full-Stack Software Developers:** Review Sections 3 (Functional Requirements), 8 (Data Dictionary), 9 (API Specifications), and 14 (Error Handling).
- **DevOps & Infrastructure Engineers:** Examine Section 4 (NFRs), Section 10 (Security), and Section 16 (Deployment Architecture).
- **Data & Machine Learning Engineers:** Refer to Section 12 (AI/ML Microservice Specification).

1.4 Product Overview & High-Level Philosophy

RentRide bridges the gap between independent vehicle owners and urban commuters. Built on a resilient three-tier architecture (React SPA, Express API, MongoDB) augmented by a Python FastAPI ML microservice, the platform enforces atomic data consistency, transparent server-side pricing, and verified digital trip handovers.

1.5 Definitions, Acronyms and Abbreviations

ACRONYM / TERM	FULL FORM & SYSTEM DEFINITION
API	Application Programming Interface: RESTful endpoints exposed by Express and FastAPI.

ACRONYM / TERM	FULL FORM & SYSTEM DEFINITION
ARAI	Automotive Research Association of India: Official certification body for vehicle fuel mileage.
Bcrypt	Adaptive cryptographic key-derivation function used for one-way password hashing.
CRUD	Create, Read, Update, Delete: Basic database manipulation operations.
DFD	Data Flow Diagram: Graphical representation of information flow through the system.
ERD	Entity-Relationship Diagram: Structural model of database entities and associations.
GST	Goods and Services Tax: Mandatory 18% statutory tax calculated on vehicle rental subtotal.
HMAC	Hash-based Message Authentication Code: Cryptographic signature used to verify Razorpay webhooks.
JWT	JSON Web Token: Compact, URL-safe means of representing claims between client and server.
KYC	Know Your Customer: Identity verification procedure validating driving license credentials.
ODM	Object Data Modeling: Mongoose abstraction layer mapping JavaScript objects to MongoDB.
RBAC	Role-Based Access Control: Security mechanism restricting resources based on user roles.
SPA	Single-Page Application: Web application loading a single HTML page (React + Vite).
TTL	Time-To-Live: Automatic expiration period used for temporary reservation locks (10 minutes).

1.6 References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
2. RentRide Architectural Repository & Technical Documentation Suite (`docs/01-15`).
3. MongoDB Inc. *Mongoose 8 Framework Guide & Multi-Index Aggregations*.
4. Razorpay Technologies. *Razorpay Standard Checkout & Webhook Signature Protocols*.

2. OVERALL DESCRIPTION

2.1 Product Perspective

RentRide operates as a sovereign, self-contained multi-vendor marketplace. It interfaces with external services strictly via authenticated HTTPS protocols (Cloudinary for media asset hosting, Razorpay for payment order capture, and internal microservice communication to FastAPI for ML inference). The system is completely cloud-agnostic, capable of local deployment via Node.js/Vite or distributed cloud containerization.

2.2 Product Functions Summary

- **User Identity & Access:** Registration, login, profile management, driving license upload, role assignment (`CUSTOMER`, `VENDOR`, `ADMIN`).
- **Marketplace Exploration:** City hub selection (Indore, Bhopal, Gwalior), multi-tier taxonomy filtering, dynamic fuel cost estimation.
- **Availability & Collision Engine:** Deterministic interval overlap conflict prevention across digital bookings, offline walk-ins, and maintenance downtime.
- **Rental Duration Flexibility:** Custom Hourly plans (1–24h), 12-Hour Day Passes, and 24-Hour Multi-day packages.
- **Checkout & Inventory Lock:** 10-minute temporary inventory reservation lock (`lockExpiresAt`).
- **Payment Reconciliation:** Razorpay order creation, payment capture, and HMAC-SHA256 verification.
- **Vendor Operations:** Fleet listing, Cloudinary photo uploads, walk-in offline bookings, digital trip handovers (odometer/OTP), and return processing.
- **Platform Governance:** Administrative vehicle moderation, user auditing, coupon lifecycle management, and global booking ledgers.
- **AI Microservice:** Multi-attribute recommendation scoring, 7-day demand forecasting, and elasticity-bounded dynamic pricing.

2.3 User Classes and Characteristics

USER CLASS	CHARACTERISTICS & TECHNICAL PROFILE	PERMISSIONS & CAPABILITIES
Public Guest	Unauthenticated visitor browsing the platform via mobile or desktop web browser.	View landing page, search and filter available vehicles, inspect vehicle details, calculate estimated fuel costs, simulate fare breakdown.
Customer	Registered commuter or traveler requiring on-demand vehicle rental services.	All Guest capabilities + create online bookings, apply coupons, execute Razorpay checkout, manage reservations, cancel trips, write verified reviews, receive in-app notifications.

USER CLASS	CHARACTERISTICS & TECHNICAL PROFILE	PERMISSIONS & CAPABILITIES
Vendor	Commercial fleet agency, bike rental shop, or vehicle owner listing fleet inventory.	Manage vendor business profile, create vehicle listings (manual or 1-click catalog auto-fill), log offline walk-in bookings, execute vehicle handover/return workflows, schedule maintenance blackouts, view earnings metrics.
Admin	System administrator responsible for platform governance and compliance.	Platform-wide superuser: approve/reject vendor vehicle listings, review user KYC status, create and deactivate promotional coupons, inspect global financial transactions, view analytics.

2.4 Operating Environment

- **Client Environment:** Modern standards-compliant web browsers (Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, Microsoft Edge 110+). Responsive on mobile (360px+), tablet, and desktop viewports.
- **Server Runtime:** Node.js version 18.x or 20.x LTS running on Linux/Windows/macOS.
- **Database Environment:** MongoDB version 6.0+ (MongoDB Atlas cloud cluster or local replica set).
- **AI Microservice Runtime:** Python 3.10+ with FastAPI, Uvicorn, Scikit-learn, Pandas, and NumPy.
- **External Network APIs:** Outbound HTTPS connectivity to `api.razorpay.com` and `api.cloudinary.com`.

2.5 Design and Implementation Constraints

- **Stateless Authentication:** All backend endpoints rely on stateless JWT tokens; no session state is maintained in server memory.
- **Mathematical Collision Consistency:** Availability queries must execute compound range checks (`$lt` / `$gt`) with strict exclusion of expired locks.
- **Deterministic Pricing Bounds:** Dynamic pricing multipliers must never fall below 0.90x or exceed 1.35x.
- **Zero Binary Storage:** The database stores no raw binary image data; all images are hosted on Cloudinary CDN and referenced by HTTPS URL and public ID.

2.6 Assumptions and Dependencies

- Customers possess an active Indian mobile number, driving license, and access to UPI or online banking via Razorpay.
- Vendors are legally authorized to lease vehicles commercially within operational jurisdictions.
- Network latency between Express backend and FastAPI microservice remains under 150ms over internal HTTP.

3. SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS

This section details the functional requirements across all 22 domain modules. Each requirement is assigned a unique identifier, priority level (High / Medium / Low), actor, preconditions, and expected behavior.

3.1 Module A: Authentication & Authorization (FR-AUTH)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-AUTH-001	The system shall register new Customer and Vendor accounts with validated name, email, password, and phone number.	User	High
FR-AUTH-002	The system shall hash passwords using Bcrypt (10 salt rounds) prior to database persistence.	System	High
FR-AUTH-003	The system shall authenticate credentials and issue a signed JSON Web Token (JWT) containing user ID and role claims.	User	High
FR-AUTH-004	The system shall enforce Role-Based Access Control (RBAC) via middleware guards (`CUSTOMER`, `VENDOR`, `ADMIN`).	System	High
FR-AUTH-005	The system shall reject requests with missing, expired, or tampered JWT signatures with HTTP 401 Unauthorized.	System	High
FR-AUTH-006	The system shall restrict client authentication attempts to 100 requests per 15-minute window via rate-limiting.	System	Medium
FR-AUTH-007	The system shall allow authenticated users to update their profile information and upload avatar images via Cloudinary.	User	Medium
FR-AUTH-008	The system shall provide a secure password change endpoint verifying the current password before updating.	User	Medium

3.2 Module B: Customer Profile & KYC Management (FR-CUST)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-CUST-001	The system shall permit customers to submit driving license credentials (number, expiry date, document URL).	Customer	High
FR-CUST-002	The system shall maintain customer KYC status flags (`PENDING`, `VERIFIED`, `REJECTED`).	System	High
FR-CUST-003	The system shall present customers with an overview of past, active, and upcoming booking reservations.	Customer	High

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-CUST-004	The system shall allow customers to cancel bookings in accordance with cancellation policy rules.	Customer	High

3.3 Module C: Vendor Onboarding & Fleets (FR-VEND)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-VEND-001	The system shall capture vendor business metadata (business name, GST number, physical address).	Vendor	High
FR-VEND-002	The system shall maintain an `isApproved` moderation flag on vendor profiles, controllable by Administrators.	Admin	High
FR-VEND-003	The system shall provide vendors with a unified dashboard summarizing fleet count, active rentals, and earnings.	Vendor	High
FR-VEND-004	The system shall calculate net vendor payouts by subtracting the platform commission (default 10%) from booking subtotals.	System	High

3.4 Module D: Vehicle Management (FR-VEH)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-VEH-001	The system shall allow vendors to list vehicles with brand, model, registration number, year, fuel, and transmission.	Vendor	High
FR-VEH-002	The system shall enforce tiered pricing definitions on vehicle creation: hourly rate, 12-hour rate, 24-hour rate.	Vendor	High
FR-VEH-003	The system shall upload vehicle gallery images to Cloudinary, storing HTTPS URLs and public IDs.	Vendor	High
FR-VEH-004	The system shall enforce status transitions across `AVAILABLE`, `RENTED`, `MAINTENANCE`, and `INACTIVE`.	System	High
FR-VEH-005	The system shall enforce unique vehicle license registration plates in uppercase format.	System	High

3.5 Module E: Vehicle Categories (FR-CAT)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-CAT-001	The system shall organize fleet vehicles into two top-level types: `two_wheeler` and `four_wheeler`.	System	High
FR-CAT-002	The system shall maintain specific category slugs: `electric-scooters`, `scooters`, `bikes`, `hatchbacks`, `sedans`, `suvs`.	System	High
FR-CAT-003	The system shall automatically generate URL slugs from category names using pre-validate Mongoose hooks.	System	Medium

3.6 Module F: City Search & Filter Engine (FR-SRCH)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-SRCH-001	The system shall provide city hub filtering supporting primary hubs (Indore, Bhopal, Gwalior).	User	High
FR-SRCH-002	The system shall filter vehicle inventory by price range, fuel type, transmission, and minimum seating capacity.	User	High
FR-SRCH-003	The system shall filter out unapproved (`isApproved: false`) or inactive vehicles from public search results.	System	High

3.7 Module G: Availability & Collision Engine (FR-AVAL)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-AVAL-001	The system shall execute mathematical interval overlap detection verifying: $(Start_{req} < End_{exist}) \wedge (End_{req} > Start_{exist})$.	System	High
FR-AVAL-002	The system shall evaluate conflicts against `CONFIRMED`, `ACTIVE`, and non-expired `PENDING_PAYMENT` bookings.	System	High
FR-AVAL-003	The system shall evaluate conflicts against active vendor `MAINTENANCE` blackout intervals.	System	High
FR-AVAL-004	The system shall exclude expired `PENDING_PAYMENT` reservations where `lockExpiresAt < Date.now()`.	System	High

3.8 Module H: Rental Duration Plans (FR-PLAN)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-PLAN-001	The system shall support three distinct rental plans: `HOURLY`, `12_HOURS`, and `24_HOURS`.	User	High
FR-PLAN-002	For `HOURLY` rentals, the system shall allow the customer to manually specify the duration between 1 and 24 integer hours.	Customer	High
FR-PLAN-003	For `12_HOURS` rentals, the system shall enforce a fixed duration of exactly 12 hours.	System	High
FR-PLAN-004	For `24_HOURS` rentals, the system shall enforce a fixed duration of exactly 24 hours.	System	High
FR-PLAN-005	The system shall reject fractional/decimal hours or durations outside [1, 24] with validation errors.	System	High

3.9 Module I: Booking Management & Reservation Locks (FR-BOOK)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-BOOK-001	The system shall generate unique human-readable booking codes in the format `RR-YYYYMMDD-XXXXX`.	System	High
FR-BOOK-002	Upon checkout initiation, the system shall assign status `PENDING_PAYMENT` with a 10-minute lock (`lockExpiresAt`).	System	High
FR-BOOK-003	The system shall prevent other customers from reserving the locked vehicle during the active 10-minute checkout window.	System	High
FR-BOOK-004	The system shall transition booking status to `CONFIRMED` upon verified Razorpay payment capture.	System	High
FR-BOOK-005	The system shall record full price breakdown audit objects within every booking document.	System	High

3.10 Module J: Server Pricing Engine (FR-PRIC)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-PRIC-001	The system shall compute base rental fees server-side, ignoring unvalidated client-submitted prices.	System	High
FR-PRIC-002	The system shall apply a 1.15x surge multiplier to base rates for rentals starting on Friday, Saturday, or Sunday.	System	High

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-PRIC-003	The system shall deduct valid coupon discounts from the surge-adjusted subtotal.	System	High
FR-PRIC-004	The system shall compute a mandatory 5% platform service fee and 18% GST on the net rental fee.	System	High
FR-PRIC-005	The system shall add the vehicle's refundable security deposit to total payable amount.	System	High

3.11 Module K: Coupons & Promotions Engine (FR-COUP)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-COUP-001	The system shall support `PERCENTAGE` and `FIXED` discount coupon types.	Admin	High
FR-COUP-002	The system shall enforce global usage quotas (`totalUsageLimit`) and individual per-user limits (`perUserLimit`).	System	High
FR-COUP-003	The system shall enforce minimum booking amount thresholds and validity start/end timestamps.	System	High

3.12 Module L: Payment Processing via Razorpay (FR-PAY)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-PAY-001	The system shall create Razorpay order instances with amount in INR currency subunit (paise).	System	High
FR-PAY-002	The system shall verify payments using HMAC-SHA256 signature verification over `order_id|payment_id`.	System	High
FR-PAY-003	The system shall record payment method, gateway order ID, payment ID, and timestamp in a `Payment` document.	System	High

3.13 Module M: Walk-In Offline Bookings (FR-WKIN)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-WKIN-001	The system shall allow vendors to log walk-in offline rentals specifying customer name, phone, and license number.	Vendor	High
FR-WKIN-002	Walk-in offline rentals shall trigger the collision engine, blocking online availability identically to online bookings.	System	High

3.14 Module N: Vehicle Handover & Return / Trip Custody (FR-TRIP)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-TRIP-001	The system shall execute `Start Trip` handover recording initial vehicle odometer reading and fuel level.	Vendor	High
FR-TRIP-002	The system shall execute `Complete Trip` return recording return odometer reading, fuel level, and return timestamp.	Vendor	High
FR-TRIP-003	The system shall calculate extra kilometer penalties if total trip distance exceeds the included daily allowance cap.	System	High

3.15 Module O: Maintenance Management (FR-MAINT)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-MAINT-001	The system shall permit vendors to schedule maintenance windows (`routine`, `repair`, `inspection`).	Vendor	High
FR-MAINT-002	The system shall automatically block vehicle availability across the scheduled maintenance time window.	System	High

3.16 Module P: Admin Governance & Moderation (FR-ADM)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-ADM-001	The system shall allow Administrators to approve or reject pending vendor vehicle listings.	Admin	High
FR-ADM-002	The system shall provide Administrators with a global ledger auditing all marketplace transactions and commissions.	Admin	High

3.17 Module Q: Customer Reviews & Ratings (FR-REV)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-REV-001	The system shall permit customers to submit verified ratings (1 to 5 stars) and text reviews only for completed trips.	Customer	Medium
FR-REV-002	The system shall automatically recalculate `ratingAverage` and `ratingCount` on the Vehicle entity via Mongoose hooks.	System	High

3.18 Module R: In-App Notifications (FR-NOTIF)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-NOTIF-001	The system shall generate in-app notification alerts upon booking confirmation, cancellation, and payment events.	System	Medium
FR-NOTIF-002	The system shall allow users to view unread alerts and mark notifications as read.	User	Medium

3.19 Module S: AI Recommendations Engine (FR-AI-REC)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-AI-REC-001	The system shall score candidate fleet vehicles against user preferences using weighted multi-attribute feature scoring.	AI Microservice	Medium
FR-AI-REC-002	The system shall normalize match scores between 60% and 99% and return ranked vehicle recommendations with highlights.	AI Microservice	Medium

3.20 Module T: AI 7-Day Demand Forecasting (FR-AI-FCST)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-AI-FCST-001	The system shall compute a 7-day demand index forecast based on city multipliers, category factors, and day seasonality.	AI Microservice	Medium
FR-AI-FCST-002	The system shall identify peak demand days and output demand classifications (`LOW`, `MEDIUM`, `HIGH`, `VERY HIGH`, `PEAK`).	AI Microservice	Medium

3.21 Module U: AI Dynamic Pricing Engine (FR-AI-PRIC)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-AI-PRIC-001	The system shall calculate a dynamic pricing multiplier bounded deterministically between 0.90x and 1.35x.	AI Microservice	Medium
FR-AI-PRIC-002	The engine shall factor day-of-week deltas, fleet scarcity thresholds (≥80% → +15%), and city deltas.	AI Microservice	Medium

3.22 Module V: Master Vehicle Catalog (FR-CATL)

REQUIREMENT ID	DESCRIPTION & SPECIFICATION	ACTOR	PRIORITY
FR-CATL-001	The system shall maintain a master database of 327 verified Indian vehicle specifications with certified ARAI mileage.	System	High
FR-CATL-002	The system shall allow vendors to select a master model and auto-populate technical specs, fuel type, and photos with 1 click.	Vendor	High

4. NON-FUNCTIONAL REQUIREMENTS

The platform enforces 12 non-functional requirements to ensure reliability, speed, and maintainability:

ID	CATEGORY	METRIC & IMPLEMENTATION REQUIREMENT
NFR-001	Performance	API response latency shall remain < 250ms for 95% of database read operations; availability collision query execution < 80ms under typical loads.
NFR-002	Security	Stateless JWT authentication with 24-hour expiration; Bcrypt 10 rounds for passwords; Helmet HTTP security headers; CORS restricted origins.
NFR-003	Reliability	The backend shall handle database disconnections gracefully, providing health status degrades without process termination.
NFR-004	Availability	Target operational uptime of 99.5% for core transactional services (search, availability, booking, payment).
NFR-005	Scalability	Stateless Node.js architecture allows horizontal scaling behind reverse proxies; MongoDB Atlas supports replica-set read scaling.
NFR-006	Maintainability	Clean separation into Layered Service Pattern (Routes → Controllers → Domain Services → Mongoose Models); strict Zod schema decoupling.
NFR-007	Usability	Mobile-first responsive UI built with Tailwind CSS; WCAG 2.1 AA compliant color contrast; clear form validation feedback.
NFR-008	Data Integrity	Compound indexing `{ vehicleId: 1, startDateTime: 1, endDateTime: 1 }` guarantees zero booking overlaps across concurrent requests.
NFR-009	Compatibility	Cross-browser compatibility across Chrome, Safari, Firefox, and Edge on desktop and mobile viewports.
NFR-010	Error Handling	Standardized error JSON contract (`ApiResponse` & `ApiError`) with HTTP status codes and user-friendly diagnostic messages.
NFR-011	Privacy	Customer driving license documents accessible only by authorized vendor and admin roles; payment card data processed directly by PCI-DSS compliant Razorpay.
NFR-012	API Consistency	Consistent RESTful JSON responses (`{ success: true, statusCode, data, message }`) across all endpoints.

5. SYSTEM ARCHITECTURE & DATA FLOW DIAGRAMS

5.1 Multi-Tier Architecture

RentRide implements a three-tier architecture augmented by an auxiliary machine learning microservice:

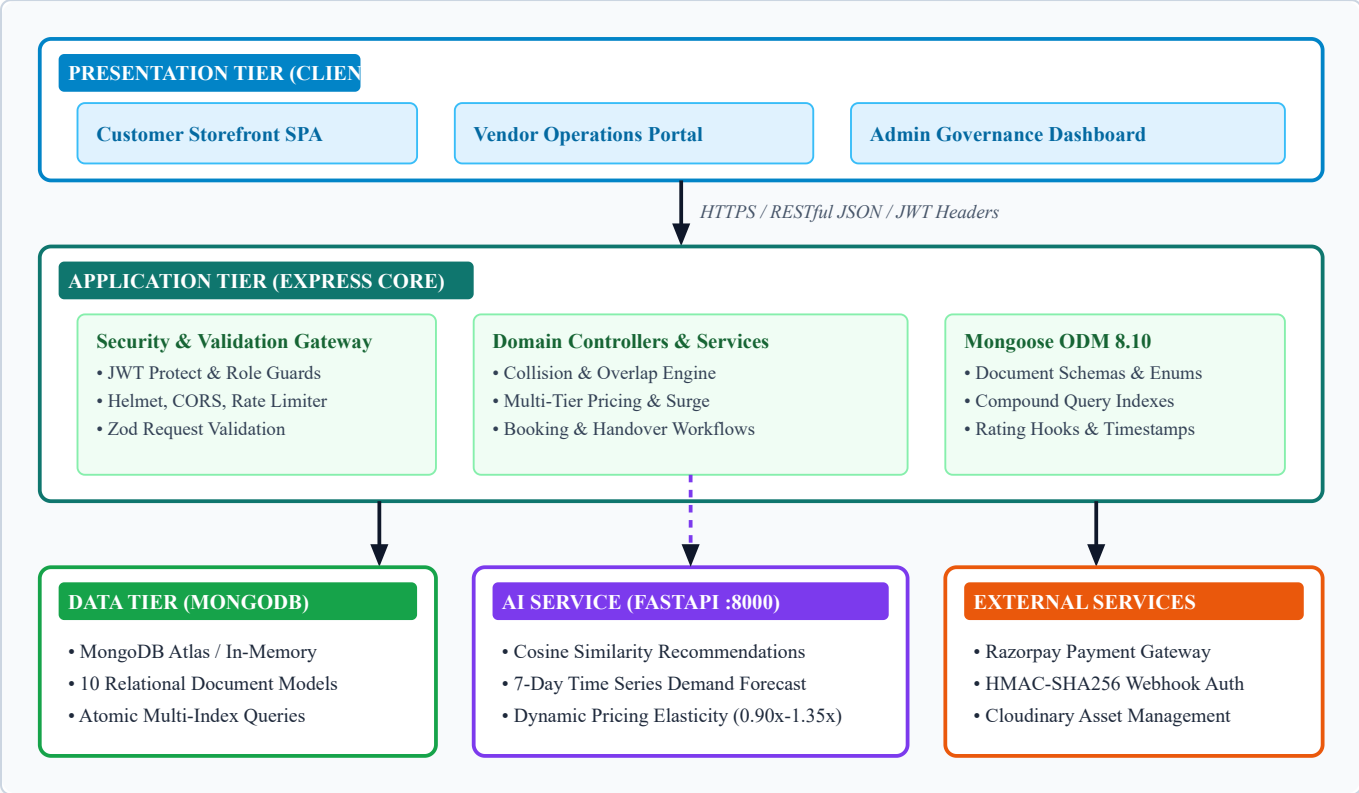


Figure 5.1: RentRide Physical & Logical Architecture

5.2 Data Flow Diagram (Level 1)

The DFD Level 1 diagram illustrates data flows between external entities (Customer, Vendor, Razorpay), core system processes (Search & Collision Engine, Booking & Pricing Engine, Handover & Return), and persistent data stores:

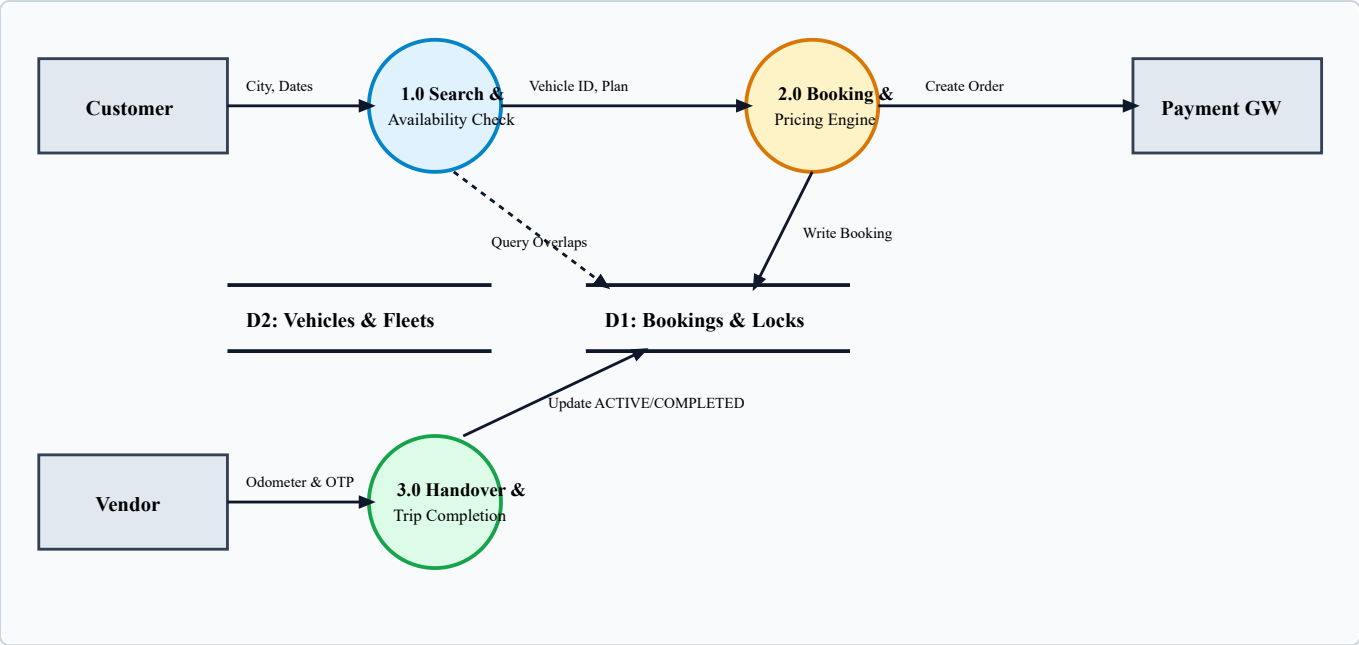


Figure 5.2: RentRide Data Flow Diagram (DFD Level 1)

6. USE CASE MODEL & SPECIFICATIONS

6.1 Use Case Diagram

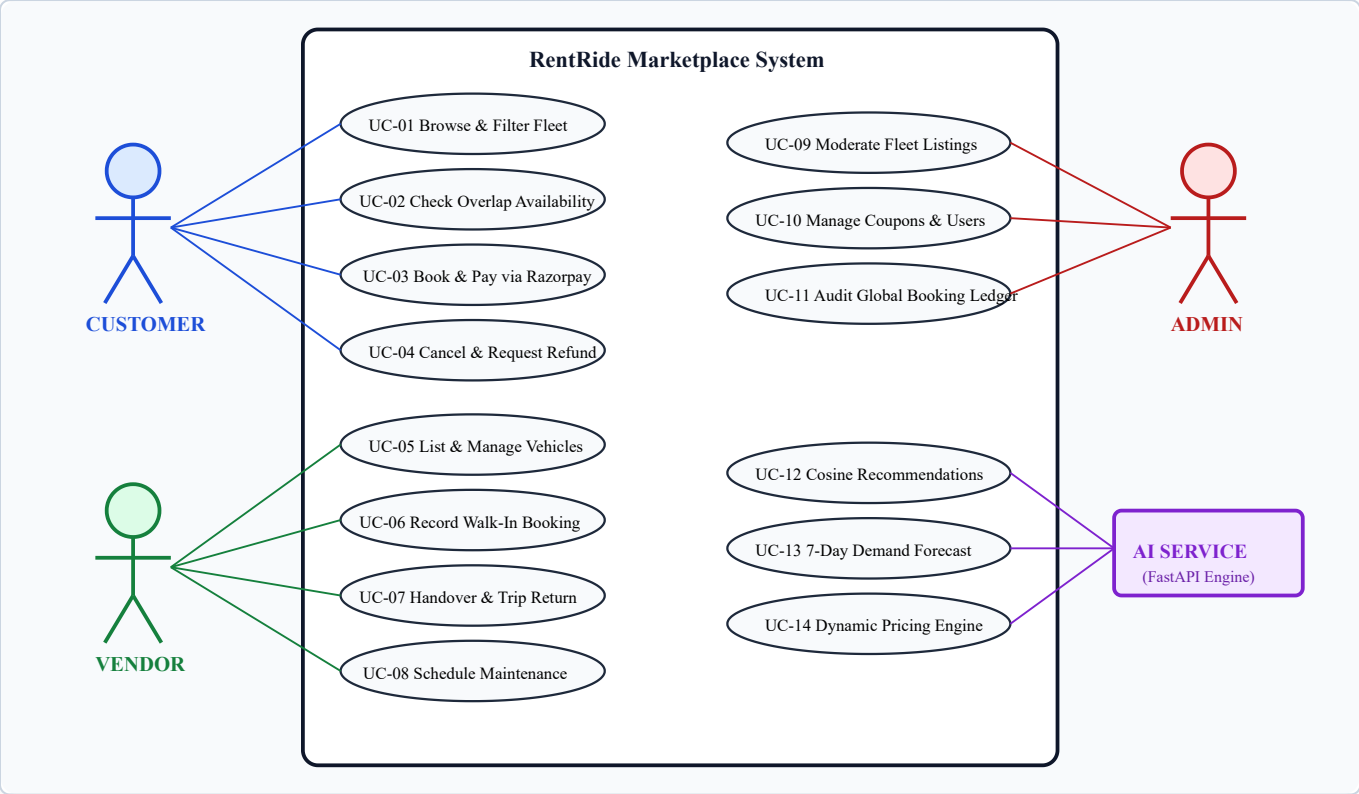


Figure 6.1: Complete Use Case Diagram

6.2 Detailed Use Case Specifications (UC-01 to UC-17)

UC-01: User Registration

- **Actor:** Public Guest / Customer / Vendor
- **Preconditions:** User provides unassigned email address.
- **Main Flow:** User submits name, email, password, phone, and role. System validates fields via Zod, hashes password via Bcrypt (10 rounds), persists User document, generates JWT, and returns authenticated session.
- **Alternative Flow:** Email already exists → System responds with HTTP 400 Bad Request.
- **Postconditions:** User record is active in MongoDB.

UC-02: User Login

- **Actor:** Customer / Vendor / Admin
- **Main Flow:** User submits email and password. System queries user including password hash, verifies password via `bcrypt.compare`, issues signed JWT, and returns role-specific profile.

- **Exception Flow:** Invalid credentials → System returns HTTP 401 Unauthorized.

UC-03: Search & Filter Vehicles

- **Actor:** Customer / Guest
- **Main Flow:** User selects city hub (Indore, Bhopal, Gwalior) and optional filters (category, price, fuel, transmission, seating). System queries approved, active vehicles matching criteria and returns vehicle cards with pricing and ratings.

UC-04: Check Vehicle Availability

- **Actor:** Customer / System
- **Main Flow:** Customer selects vehicle, rental plan, duration, and start timestamp. System executes mathematical interval overlap check. If no overlapping bookings or maintenance intervals exist, system returns available status with exact authoritative price quote.
- **Alternative Flow:** Overlap detected → System returns unavailable status with conflicting window notice.

UC-05: Create Online Booking & 10-Minute Lock

- **Actor:** Customer
- **Preconditions:** Vehicle is available for requested interval; customer is authenticated.
- **Main Flow:** Customer confirms schedule. System creates Booking with status `PENDING_PAYMENT`, computes server-side price breakdown, and assigns `lockExpiresAt = Date.now() + 10 minutes`. Vehicle is locked from concurrent reservations.

UC-06: Make Payment (Razorpay Checkout)

- **Actor:** Customer / Payment Gateway
- **Main Flow:** Customer invokes Razorpay modal for locked booking. Upon successful transaction, client returns `razorpay_order_id`, `razorpay_payment_id`, and `razorpay_signature`. Backend verifies HMAC-SHA256 signature, transitions booking status to `CONFIRMED`, and creates audited Payment record.
- **Exception Flow:** Signature mismatch → Payment marked `FAILED`, booking remains unconfirmed.

UC-07: Cancel Booking

- **Actor:** Customer / Vendor / Admin
- **Main Flow:** Customer requests cancellation on confirmed booking. System verifies cancellation policy, computes refund amount and cancellation fee, transitions status to `CANCELLED`, and releases inventory slot.

UC-08: Vendor Adds Vehicle

- **Actor:** Vendor
- **Main Flow:** Vendor submits vehicle specifications, tiered pricing (hourly, 12h, 24h), and photographs. System uploads images to Cloudinary, sets `isApproved = true` (or pending moderation), and saves vehicle record.

UC-09: Vendor Logs Walk-In Offline Booking

- **Actor:** Vendor
- **Main Flow:** Vendor inputs walk-in customer details (name, phone, license) and dates. System runs collision check and saves Booking with `bookingType = OFFLINE`, instantly blocking online marketplace availability.

UC-10: Vehicle Handover (Start Trip)

- **Actor:** Vendor
- **Main Flow:** Customer arrives at pickup hub. Vendor records starting odometer reading and fuel level. System transitions booking status from `CONFIRMED` to `ACTIVE`.

UC-11: Vehicle Return (Complete Trip)

- **Actor:** Vendor
- **Main Flow:** Customer returns vehicle. Vendor inputs return odometer reading. System computes extra km driven against daily cap, adds extra km fee if applicable, releases security deposit, and marks booking `COMPLETED`.

UC-12: Schedule Maintenance

- **Actor:** Vendor
- **Main Flow:** Vendor inputs service window (`routine`, `repair`, `inspection`). System verifies no confirmed bookings conflict with the window, creates Maintenance record, and blocks marketplace availability.

UC-13: Admin Approves/Rejects Vehicle

- **Actor:** Admin
- **Main Flow:** Admin inspects submitted vendor vehicle listing and clicks Approve or Reject. System updates `isApproved` flag, making vehicle visible or hidden in public marketplace.

UC-14: Submit Verified Review

- **Actor:** Customer
- **Preconditions:** Booking status is `COMPLETED`; customer has not already reviewed this booking.
- **Main Flow:** Customer submits star rating (1-5) and comment. System saves Review, triggers Mongoose post-save hook, and recalculates vehicle's `ratingAverage` and `ratingCount`.

UC-15: AI Vehicle Recommendation

- **Actor:** Customer / AI Microservice
- **Main Flow:** System passes customer preferences to FastAPI `/recommendations`. Microservice evaluates candidate fleet via multi-attribute scoring, returns ranked recommendations with percentage match scores (60%–99%).

UC-16: AI 7-Day Demand Forecasting

- **Actor:** Vendor / Admin / AI Microservice
- **Main Flow:** Vendor views demand forecast for city hub. Microservice analyzes day-of-week seasonality, city multipliers, and fleet utilization, outputting a 7-day demand index curve.

UC-17: AI Dynamic Pricing Multiplier

- **Actor:** System / AI Microservice
- **Main Flow:** Pricing engine requests dynamic rate multiplier for vehicle booking date. Microservice applies day-of-week deltas, fleet scarcity thresholds, and city factors, returning multiplier bounded in [0.90, 1.35].

7. DATA REQUIREMENTS & ENTITY-RELATIONSHIP MODEL

RentRide implements 10 normalized document models on MongoDB Atlas. The Entity-Relationship Model demonstrates the structural associations:

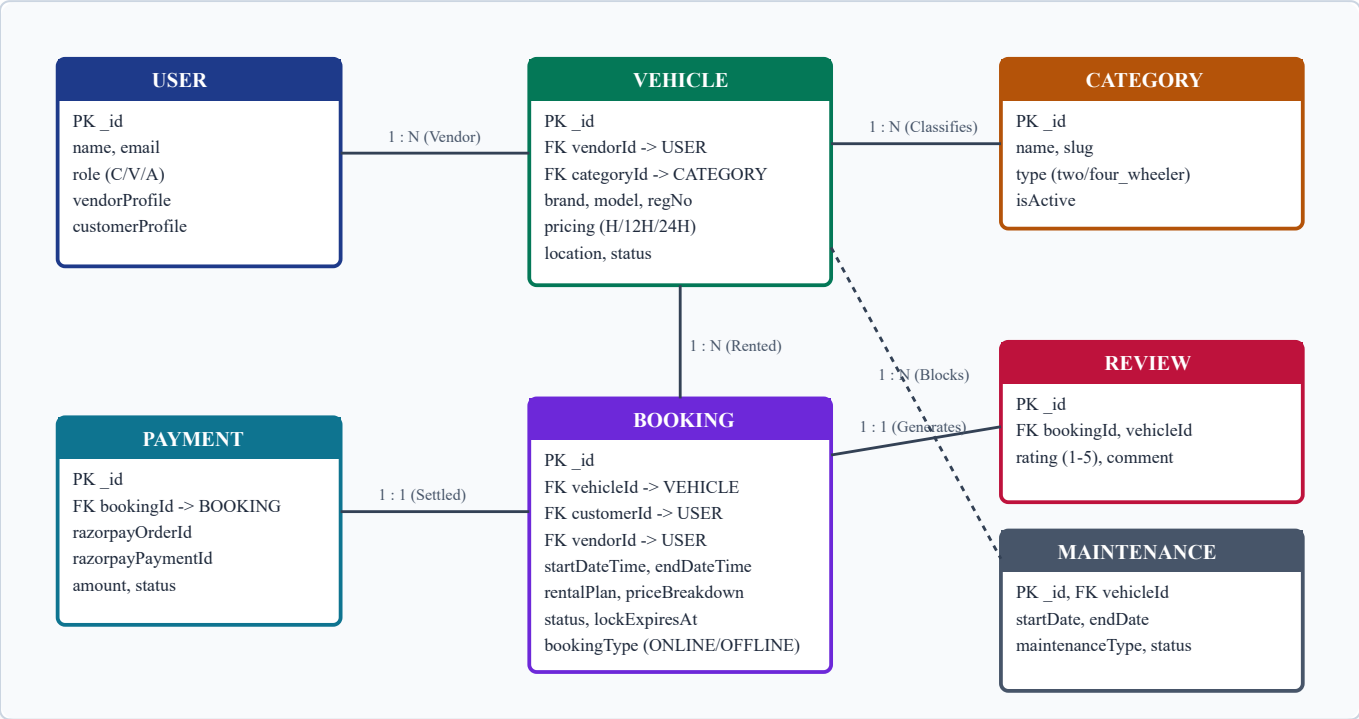


Figure 7.1: RentRide Entity-Relationship Diagram (ERD)

7.1 Entity Cardinalities & Multiplicity Rules

- **User (Vendor) to Vehicle:** One-to-Many (1 : N). A vendor owns multiple vehicles; each vehicle has exactly one vendor owner.
- **Category to Vehicle:** One-to-Many (1 : N). A category classifies many vehicles; each vehicle references one category.
- **Vehicle to Booking:** One-to-Many (1 : N). A vehicle is associated with multiple historical and upcoming bookings.
- **Customer to Booking:** One-to-Many (1 : N). A customer creates multiple bookings across their account lifecycle.
- **Booking to Payment:** One-to-One (1 : 1). Each online booking references exactly one authoritative payment transaction.
- **Booking to Review:** Zero-to-One (0 : 1). A completed booking produces at most one verified customer review.
- **Vehicle to Maintenance:** One-to-Many (1 : N). A vehicle undergoes multiple scheduled maintenance events.

8. DATA DICTIONARY

The data dictionary specifies primary field attributes, types, nullability, and validation constraints for core entities:

8.1 User Entity (`users`)

FIELD NAME	DATA TYPE	REQ	DESCRIPTION & CONSTRAINTS
_id	ObjectId	Yes	Primary Key (auto-generated by MongoDB).
name	String	Yes	User full legal name (min 2, max 60 characters).
email	String	Yes	Unique lowercase email address with RFC regex validation.
password	String	Yes	Hashed password via Bcrypt (10 rounds); `select: false`.
phone	String	No	Contact phone number string.
role	String	Yes	Enum: `['CUSTOMER', 'VENDOR', 'ADMIN']` (default: `'CUSTOMER'`).
vendorProfile.businessName	String	No	Registered trade name for commercial vendors.
vendorProfile.gstNumber	String	No	15-character Goods and Services Tax ID.
vendorProfile.isApproved	Boolean	Yes	Admin verification status (default: `false`).
customerProfile.drivingLicenseNumber	String	No	Driving license ID for customer trip custody.
customerProfile.kycStatus	String	Yes	Enum: `['PENDING', 'VERIFIED', 'REJECTED']` (default: `'PENDING'`).

8.2 Vehicle Entity (`vehicles`)

FIELD NAME	DATA TYPE	REQ	DESCRIPTION & CONSTRAINTS
_id	ObjectId	Yes	Primary Key.
vendorId	ObjectId	Yes	Foreign Key referencing `User._id`.
categoryId	ObjectId	Yes	Foreign Key referencing `Category._id`.
brand	String	Yes	Manufacturer name (e.g., 'Royal Enfield', 'Tata').
model	String	Yes	Model designation (e.g., 'Hunter 350', 'Nexon EV').
registrationNumber	String	Yes	Unique uppercase license plate number.
fuelType	String	Yes	Enum: `['petrol', 'diesel', 'electric', 'cng', 'hybrid']`.

FIELD NAME	DATA TYPE	REQ	DESCRIPTION & CONSTRAINTS
transmission	String	Yes	Enum: `['manual', 'automatic']`.
seatingCapacity	Number	Yes	Passenger capacity (minimum 1).
location.city	String	Yes	Operating city hub (e.g., 'Indore', 'Bhopal', 'Gwalior').
pricing.hourlyRate	Number	Yes	Base rate per hour in INR (min ₹10).
pricing.rate12Hour	Number	Yes	Fixed rate for 12-hour day pack in INR (min ₹50).
pricing.rate24Hour	Number	Yes	Fixed rate for 24-hour day pack in INR (min ₹100).
status	String	Yes	Enum: `['AVAILABLE', 'RENTED', 'MAINTENANCE', 'INACTIVE']`.
ratingAverage	Number	No	Aggregate rating from 0.0 to 5.0 (default: 0).

8.3 Booking Entity (`bookings`)

FIELD NAME	DATA TYPE	REQ	DESCRIPTION & CONSTRAINTS
_id	ObjectId	Yes	Primary Key.
bookingNumber	String	Yes	Unique code: `RR-YYYYMMDD-XXXXX`.
vehicleId	ObjectId	Yes	Foreign Key referencing `Vehicle._id`.
customerId	ObjectId	No	Foreign Key referencing `User._id` (null for offline walk-ins).
rentalPlan	String	Yes	Enum: `['HOURLY', '12_HOURS', '24_HOURS']`.
startDateTime	Date	Yes	Rental reservation start timestamp.
endDateTime	Date	Yes	Rental reservation end timestamp.
durationHours	Number	Yes	Total billable rental hours.
priceBreakdown.totalAmount	Number	Yes	Total gross payable amount by customer in INR.
priceBreakdown.vendorPayout	Number	Yes	Net earnings payable to fleet vendor.
status	String	Yes	Enum: `['PENDING_PAYMENT', 'CONFIRMED', 'ACTIVE', 'COMPLETED', 'CANCELLED', 'EXPIRED']`.
lockExpiresAt	Date	Yes	Temporary 10-minute reservation lock expiration timestamp.

9. REST API SPECIFICATIONS

The Express application exposes 14 controller route domains. All endpoints return standardized JSON structures:

HTTP METHOD	ROUTE PATH	AUTH & ROLE	PURPOSE & PAYLOAD DESCRIPTION
POST	/api/auth/register	Public	Register new customer or vendor user account.
POST	/api/auth/login	Public	Authenticate credentials; returns JWT and user profile.
GET	/api/auth/me	Protected	Retrieve current authenticated user session data.
GET	/api/vehicles	Public	Search fleet with city, category, price, and fuel filters.
GET	/api/vehicles/:id	Public	Retrieve complete vehicle details and vendor hub information.
POST	/api/bookings/check-availability	Public	Dry-run interval overlap test and authoritative price quote.
POST	/api/bookings/online	Customer	Create reservation with status `PENDING_PAYMENT` (10m lock).
GET	/api/bookings/my-bookings	Customer	Retrieve customer's historical and active bookings.
POST	/api/bookings/:id/cancel	Protected	Execute booking cancellation and compute refund amount.
POST	/api/payments/create-order	Customer	Initialize Razorpay order instance for booking amount.
POST	/api/payments/verify	Customer	Verify HMAC-SHA256 signature; confirms booking.
GET	/api/vendor/dashboard	Vendor	Retrieve vendor KPI metrics, active trips, and fleet count.
POST	/api/vendor/vehicles	Vendor	Create new vehicle listing with Cloudinary image upload.
POST	/api/vendor/bookings/offline	Vendor	Log walk-in customer booking, blocking marketplace slot.
POST	/api/trips/:bookingId/handover	Vendor	Execute Start Trip handover with initial odometer reading.
POST	/api/trips/:bookingId/return	Vendor	Execute Complete Trip return, calculating extra km charges.
POST	/api/maintenance	Vendor	Schedule maintenance window, blacking out vehicle dates.
GET	/api/admin/vehicles/pending	Admin	Fetch unmoderated vehicle listings awaiting approval.

HTTP METHOD	ROUTE PATH	AUTH & ROLE	PURPOSE & PAYLOAD DESCRIPTION
PATCH	/api/admin/vehicles/:id/status	Admin	Approve or reject vendor vehicle listing.
POST	/api/fuel-prices/estimate	Public	Compute estimated fuel required and cost from distance.
POST	/api/ai/recommendations	Public	FastAPI proxy for multi-attribute vehicle recommendations.
GET	/api/ai/demand-forecast	Public	FastAPI proxy for 7-day city hub demand forecasting.

10. SECURITY REQUIREMENTS & IMPLEMENTATION

Security is enforced across the presentation, network, application, and data layers:

10.1 Authentication & Token Architecture

- **JWT Algorithm:** HMAC-SHA256 signing using a 256-bit environment secret (`JWT_SECRET`). Tokens carry subject `id` and claim `role`.
- **Token Expiration:** Default 24-hour lifetime, preventing indefinite session exposure.
- **HttpOnly Cookie Storage:** Tokens can be transmitted via `Authorization: Bearer <token>` header or partitioned HttpOnly cookies to mitigate XSS exposure.

10.2 Cryptographic Password Protection

Passwords undergo one-way adaptive salting and hashing via `bcryptjs` with a work factor of 10. Cleartext passwords are never logged, transmitted in error traces, or stored in database backups.

10.3 Web Application Firewalls & Middlewares

- **Helmet:** Sets HTTP response headers (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Strict-Transport-Security`).
- **CORS:** Whitelists explicit origin origins matching `CLIENT_URL` with credentials support.
- **Rate Limiting:** Enforces `express-rate-limit` restricting client IPs to 100 requests per 15-minute window.
- **Zod Input Sanitization:** Request payloads undergo strict validation against Zod schemas; unexpected fields are automatically stripped.

11. BOOKING & BUSINESS LOGIC SPECIFICATION

11.1 Mathematical Interval Overlap Collision Prevention

RentRide enforces deterministic interval mathematics to prevent double-booking. For any candidate booking request with requested interval [$Start_{req}$, End_{req}] and existing vehicle interval [$Start_{exist}$, End_{exist}], a collision occurs if and only if:

$$Collision \Leftrightarrow (Start_{req} < End_{exist}) \wedge (End_{req} > Start_{exist})$$

The database executes this check atomically using compound indexes:

```
const conflictingBooking = await Booking.findOne({
  vehicleId,
  status: { $in: ['CONFIRMED', 'ACTIVE', 'PENDING_PAYMENT'] },
  startDateTime: { $lt: requestedEnd },
  endDateTime: { $gt: requestedStart },
  $or: [
    { status: { $in: ['CONFIRMED', 'ACTIVE'] } },
    { status: 'PENDING_PAYMENT', lockExpiresAt: { $gt: new Date() } }
  ]
});
```

11.2 10-Minute Temporary Reservation Lock

When a customer clicks "Reserve Vehicle", the booking is created with status `PENDING_PAYMENT` and `lockExpiresAt = Date.now() + 10m`. During these 10 minutes, no other user can book the vehicle. If payment succeeds, status updates to `CONFIRMED`. If the customer abandons checkout, queries automatically ignore the lock once `lockExpiresAt < Date.now()`.

11.3 Booking Lifecycle State Machine

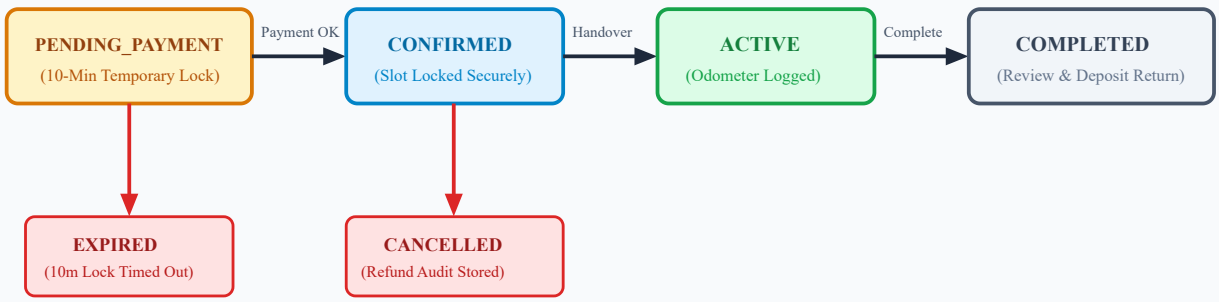


Figure 11.1: Booking State Transition Lifecycle Diagram

11.4 Pricing Formulation

The server computes transparent pricing breakdowns using exact formulas:

$$\text{Base Price} = \text{HourlyRate} \times \text{DurationHours (or Rate12Hour / Rate24Hour for fixed plans)}$$

$$\text{Surge Adjusted} = \text{round}(\text{Base Price} \times \text{SurgeMultiplier}) \text{ [Surge = 1.15 on Fri/Sat/Sun]}$$

$$\text{Subtotal} = \max(0, \text{Surge Adjusted} - \text{CouponDiscount})$$

$$\text{Platform Fee} = \text{round}(0.05 \times \text{Subtotal})$$

$$\text{GST} = \text{round}(0.18 \times \text{Subtotal})$$

$$\text{Total Payable} = \text{Subtotal} + \text{Platform Fee} + \text{GST} + \text{Security Deposit}$$

$$\text{Vendor Payout} = \text{Subtotal} \times (1 - \text{CommissionRate}) \text{ [Default Commission = 10\%]}$$

12. AI / ML MICROSERVICE DETAILED SPECIFICATION

The AI subsystem is deployed as an asynchronous Python/FastAPI microservice. It provides three core services:

12.1 Recommendation Scoring Engine

Computes multi-attribute match scores between vehicle vectors and customer preference vectors:

- **Category Match:** +25 points for exact category slug match.
- **Budget Compatibility:** +20 points if hourly rate ≤ maxBudget; penalty proportional to excess ratio if above.
- **Passenger Capacity:** +15 points if seating capacity ≥ requested passengers (−10 if less).
- **Engine & Transmission:** +10 points for matching fuel type; +10 points for matching transmission.
- **Rating Boost:** Scaled boost: (ratingAverage − 3.0) × 6.0.
- **Output:** Normalized score bounded between 60% and 99% with human-readable recommendation highlights.

12.2 7-Day Demand Forecaster

Analyzes day-of-week seasonality, city multipliers, and fleet utilization pressure:

DAY OF WEEK	DAY FACTOR	OPERATIONAL INTERPRETATION
Monday	0.85	Post-weekend drop in leisure demand.
Tuesday	0.80	Lowest weekly demand trough (Value Saver discount day).
Wednesday	0.90	Mid-week baseline commuter demand.
Thursday	1.05	Pre-weekend booking inquiries begin.
Friday	1.45	Significant surge in weekend trip departures.
Saturday	1.60	Peak weekly demand period across all hubs.
Sunday	1.35	Sustained weekend leisure and return rentals.

12.3 Dynamic Pricing Engine

Computes dynamic rate multipliers bounded between 0.90x and 1.35x:

$$\textit{Dynamic Multiplier} = \textit{clamp}(0.90, 1.35, \textit{BaseMultiplier} + \textit{DayDelta} + \textit{ScarcityDelta} + \textit{CityDelta})$$

- Saturday: +0.15 | Friday/Sunday: +0.10 | Tuesday/Wednesday: −0.05
- Fleet Utilization ≥ 80%: +0.15 | ≥ 65%: +0.08 | < 35%: −0.05

- City Multipliers: Indore (+0.05), Bhopal (+0.03), Gwalior (+0.02)

13. USER INTERFACE SPECIFICATIONS

RentRide implements a design system with dedicated user layouts:

- **Customer Storefront:** Minimalist city selector, multi-parameter filter drawer, grid/card vehicle listings with ARAI mileage tags, sticky `FareCalculator` drawer on details view, and checkout modal.
- **Vendor Operations Portal:** Collapsible sidebar navigation, live fleet inventory table, walk-in offline booking modal, and trip custody action buttons (`Start Trip`, `Complete Trip`).
- **Admin Governance Dashboard:** Tabbed management shell for vehicle listing approvals, coupon creation, user account moderation, and global financial audit metrics.
- **Design System Tokens:** Primary Brand Indigo (`#4f46e5`), Emerald Accent (`#10b981`), Amber Warnings (`#f59e0b`), Slate Dark Mode (`#0f172a`), Typography in Outfit (headings) and Inter (body).

14. ERROR HANDLING & EXCEPTION MATRIX

ERROR CONDITION	HTTP CODE	SYSTEM RESPONSE & CLIENT MITIGATION
Validation Error	400 Bad Request	Zod returns formatted array of field errors (`{ field, message }`).
Authentication Missing	401 Unauthorized	Client redirects to `/login` with return URL query param.
Forbidden Role Action	403 Forbidden	User role lacks required permissions; access denied banner shown.
Vehicle Slot Collision	409 Conflict	Availability conflict returned; suggestions for alternate slots shown.
Resource Not Found	404 Not Found	Returns descriptive missing entity message.
Rate Limit Exceeded	429 Too Many Req	Express-rate-limit blocks traffic; retry-after header sent.
Internal Server Error	500 Server Error	Error logged via Winston/console; sanitized client message returned.

15. TESTING REQUIREMENTS & VERIFICATION SUITE

The platform enforces comprehensive testing protocols across critical operational paths:

- **Booking Duration Flow Verification (`testBookingDurationFlow.js`):** 20 automated tests validating:
  - Hourly plan: 1h, 2h, 6h, 12h, 24h accepted; 0h, negative, decimal (2.5h), and >24h strictly rejected.
  - 12-Hour plan: exactly 12 hours accepted; non-12h rejected.
  - 24-Hour plan: exactly 24 hours accepted; non-24h rejected.
  - Pricing service zero-trust validation: verifies base price calculations across all plans.
- **Fuel Cost Dynamic Calculator Verification (`testFuelCostCalculation.js`):** 13 automated tests verifying:
  - Test 1: Distance 150 km, Mileage 45 km/l, Price ₹100/l → 3.33 L, ₹333.
  - Test 2: Distance 50 km → 1.11 L, ₹111.
  - Test 3: Distance 100 km → 2.22 L, ₹222.
  - Test 4: Distance 0 km → 0 L, ₹0 (no NaN/Infinity).
  - Test 5: Empty distance → status `EMPTY` with helper message.
  - Test 6–7: Negative and string inputs rejected.
  - Test 8–11: Multi-vehicle comparisons, Diesel (L), CNG (kg), EV (kWh), and backend parity.
- **Dynamic Pricing Engine Verification (`test_pricing.py`):** 24 automated tests in Python verifying deterministic bounds [0.90x, 1.35x], day deltas, and scarcity multipliers.

16. DEPLOYMENT & INFRASTRUCTURE

RentRide is engineered for flexible deployment across local development and production environments:

- **Local Development Topology:** Vite HMR frontend (Port 5173), Express API (Port 5000), FastAPI AI service (Port 8000), with MongoDB Atlas or local `mongodb-memory-server` fallback.
- **Production Cloud Topology:** React SPA served via Cloudflare CDN / Vercel; Express backend containerized on AWS ECS / Render; FastAPI microservice deployed on container instances; MongoDB Atlas dedicated M10 cluster with automated daily snapshots; Cloudinary CDN media delivery.

17. SYSTEM LIMITATIONS & KNOWN CONSTRAINTS

In accordance with academic rigor, current system limitations are explicitly documented:

1. **Automated Razorpay Refund API Trigger:** While cancellation fees and refund amounts are computed and saved in `cancellationDetails`, live API dispatch to `razorpay.payments.refund()` is not yet executed automatically.
2. **Customer KYC Verification UI:** Driving license fields are stored and displayed, but admin verification action buttons currently operate as a status viewer.
3. **Email and SMS Dispatch:** Transactional vouchers are generated in-app; SMTP (Nodemailer) and SMS gateway (Twilio) dispatch are not configured in the active build.
4. **Interactive Map UI:** Geographic coordinates (`{ lat, lng }`) are stored for pickup hubs, but interactive Leaflet map pins are not yet mounted on the Explore page.
5. **Expired Lock Cron Worker:** Expired locks are excluded in availability queries, but background cron worker to transition status from `PENDING_PAYMENT` to `EXPIRED` is pending implementation.

18. DEVELOPMENT ROADMAP & FUTURE SCOPE

- **Phase 1 (Operational Hardening):** Live Razorpay refund dispatch, background TTL lock sweeper, admin KYC approval buttons.
- **Phase 2 (Communications & Maps):** Email/SMS voucher dispatch with QR codes, interactive Leaflet/Google Maps on Explore view, vendor payout settlement ledger.
- **Phase 3 (Telematics & AI Retraining):** Live OBD-II and GPS telematics ingestion for speed and EV battery tracking; automated weekly retraining for demand models.
- **Phase 4 (Enterprise Caching):** Redis caching layer with 5-minute TTL for public vehicle catalog queries.
- **Phase 5 (Containerization):** Multi-stage Docker containerization with `docker-compose.yml` and Cloudflare edge ingress.

19. CONCLUSION & SIGNOFF

This Software Requirements Specification establishes the complete, authoritative engineering baseline for the **RentRide Multi-Vendor Vehicle Rental Marketplace**. By defining rigorous functional requirements, mathematical collision checks, tiered rental plans, robust security constraints, and predictive machine learning models, this specification serves as the blueprint for development, verification, and academic submission.

Prepared By:	**Approved By:**
_____	_____
Student Candidate (B.Tech CSE)	Project Guide / Academic Supervisor